

-Kabir Sinha

-E23CSEU1343

CSET419 – Introduction to Generative AI

Lab 9: Sequence Generation using LSTM and Transformer Models

```
[1]  
✓ 4s      import torch  
          import torch.nn as nn  
          import numpy as np
```

[2]

✓ Os

```
text = """
machine learning models learn patterns from data.
sequence models process data step by step.
recurrent neural networks are designed for sequential tasks.
rnn models maintain hidden states across time steps.

long short term memory networks solve long dependency problems.
lstm uses gates to control information flow.
gru models simplify the lstm architecture.
sequence prediction is useful in many applications.

language modeling predicts the next word in a sentence.
speech recognition processes audio sequences.
time series forecasting predicts future values.
music generation creates new melodies.

generative models learn probability distributions.
they generate new samples similar to training data.
sequence generation is widely used in artificial intelligence.
deep learning improves sequence modeling performance.
"""
```

[3]

✓ Os

```
words = text.lower().split()
vocab = sorted(set(words))

word2idx = {w:i for i,w in enumerate(vocab)}
idx2word = {i:w for w,i in word2idx.items()}

encoded = [word2idx[w] for w in words]

seq_length = 5
X = []
y = []

for i in range(len(encoded)-seq_length):
    X.append(encoded[i:i+seq_length])
    y.append(encoded[i+seq_length])

X = torch.tensor(X)
y = torch.tensor(y)
```

[4]

✓ 0s

```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embed_size=64, hidden_size=128):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out

model = LSTMModel(len(vocab))
```

[5]

✓ 6s

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

epochs = 200

for epoch in range(epochs):
    outputs = model(X)
    loss = criterion(outputs, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

▼

```
Epoch 0, Loss: 4.4767
Epoch 50, Loss: 0.0004
Epoch 100, Loss: 0.0002
Epoch 150, Loss: 0.0002
```

[6]

✓ Os

```
def generate_text(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()

    for _ in range(length):
        x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])
        pred = model(x)
        next_word = idx2word[pred.argmax().item()]
        words_seed.append(next_word)

    return " ".join(words_seed)

print(generate_text(model, "machine learning models"))
```

machine learning models maintain hidden states across time steps. long short term memory

[7]

✓ Os

```
class SimpleTransformer(nn.Module):
    def __init__(self, vocab_size, embed_size=64, heads=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.attention = nn.MultiheadAttention(embed_size, heads, batch_first=True)
        self.fc = nn.Linear(embed_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        attn_output, _ = self.attention(x, x, x)
        out = self.fc(attn_output[:, -1, :])
        return out

transformer = SimpleTransformer(len(vocab))
```

[8]

✓ Os

```
optimizer = torch.optim.Adam(transformer.parameters(), lr=0.01)

for epoch in range(200):
    outputs = transformer(X)
    loss = criterion(outputs, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

✓

```
Epoch 0, Loss: 4.5007
Epoch 50, Loss: 0.0135
Epoch 100, Loss: 0.0132
Epoch 150, Loss: 0.0131
```

```
[9]
✓ Os ▶ def generate_text_tf(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()

    for _ in range(length):
        x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])
        pred = model(x)
        next_word = idx2word[pred.argmax().item()]
        words_seed.append(next_word)

    return " ".join(words_seed)

print(generate_text_tf(transformer, "sequence models process"))

... sequence models process data step by step. recurrent neural networks are designed for
```

import torch

import torch.nn as nn

text = ""

machine learning models learn patterns from data.

sequence models process data step by step.

recurrent neural networks are designed for sequential tasks.

rnn models maintain hidden states across time steps.

long short term memory networks solve long dependency problems.

lstm uses gates to control information flow.

gru models simplify the lstm architecture.

sequence prediction is useful in many applications.

language modeling predicts the next word in a sentence.

speech recognition processes audio sequences.

time series forecasting predicts future values.

music generation creates new melodies.

generative models learn probability distributions.

they generate new samples similar to training data.

sequence generation is widely used in artificial intelligence.

deep learning improves sequence modeling performance.

"""

```
words = text.lower().split()
```

```
vocab = sorted(set(words))
```

```
word2idx = {w:i for i,w in enumerate(vocab)}
```

```
idx2word = {i:w for w,i in word2idx.items()}
```

```
encoded = [word2idx[w] for w in words]
```

```
seq_length = 5
```

```
X = []
```

```
y = []
```

```
for i in range(len(encoded)-seq_length):
```

```
    X.append(encoded[i:i+seq_length])
```

```
y.append(encoded[i+seq_length])
```

```
X = torch.tensor(X)
```

```
y = torch.tensor(y)
```

```
class LSTMModel(nn.Module):
```

```
    def __init__(self, vocab_size, embed_size=64, hidden_size=128):
```

```
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embed_size)
```

```
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
```

```
        self.fc = nn.Linear(hidden_size, vocab_size)
```

```
    def forward(self, x):
```

```
        x = self.embedding(x)
```

```
        out, _ = self.lstm(x)
```

```
        out = self.fc(out[:, -1, :])
```

```
        return out
```

```
model = LSTMModel(len(vocab))
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

```
for epoch in range(200):
```

```
outputs = model(X)
loss = criterion(outputs, y)
```

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

```
def generate_text(model, seed, length=10):
    model.eval()
    words_seed = seed.lower().split()

    for _ in range(length):
        x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])
        pred = model(x)
        next_word = idx2word[pred.argmax().item()]
        words_seed.append(next_word)

    return " ".join(words_seed)
```

```
print(generate_text(model, "machine learning models"))
```

```
class SimpleTransformer(nn.Module):
    def __init__(self, vocab_size, embed_size=64, heads=2):
```

```
super().__init__()
self.embedding = nn.Embedding(vocab_size, embed_size)
self.attention = nn.MultiheadAttention(embed_size, heads, batch_first=True)
self.fc = nn.Linear(embed_size, vocab_size)
```

```
def forward(self, x):
    x = self.embedding(x)
    attn_output, _ = self.attention(x, x, x)
    out = self.fc(attn_output[:, -1, :])
    return out
```

```
transformer = SimpleTransformer(len(vocab))
```

```
optimizer = torch.optim.Adam(transformer.parameters(), lr=0.01)
```

```
for epoch in range(200):
    outputs = transformer(X)
    loss = criterion(outputs, y)
```

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

```
def generate_text_tf(model, seed, length=10):
```

```
model.eval()
```

```
words_seed = seed.lower().split()
```

```
for _ in range(length):
```

```
    x = torch.tensor([[word2idx[w] for w in words_seed[-seq_length:]]])
```

```
    pred = model(x)
```

```
    next_word = idx2word[pred.argmax().item()]
```

```
    words_seed.append(next_word)
```

```
return " ".join(words_seed)
```

```
print(generate_text_tf(transformer, "sequence models process"))
```